Bags, Vectors & Transformers

Neural Networks, the Basics · Bonus, Part 6

Denise J. Roth, PhD Candidate
Strategic Communication Group · Wageningen University & Research

Why Look Under the Hood?

Every model from Day 3 onward (BERT, sentence transformers, LLMs) is a **neural network**. You can use them without knowing how they work, but a little understanding goes a long way:

Settings make sense

Learning rate, epochs, batch size, dropout: you'll know what each one does and why it matters.

You can diagnose

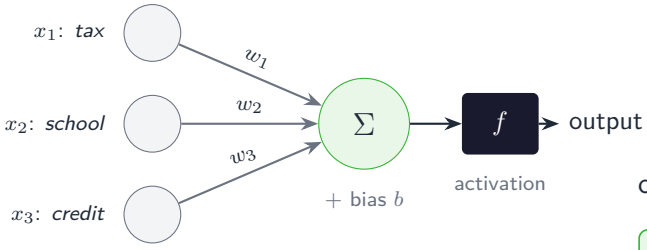
Is the model still learning, or memorizing its training data? The learning curves tell you.

You choose wisely

When does a neural network beat logistic regression, and when is it just a slower route to the same result?

This bonus module comes with a notebook that builds the same network in scikit-learn, Keras and PyTorch.

One Neuron

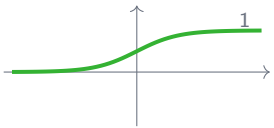


1. Multiply each input by a **weight**
2. Add them up, plus a **bias**
3. Pass the sum through an **activation function**

$$\text{output} = f(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

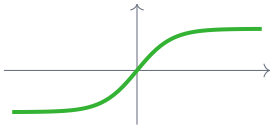
With a sigmoid as f, one neuron is exactly **logistic regression** from Day 1.

Activation Functions



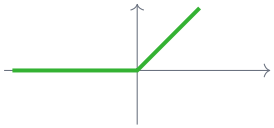
sigmoid

between 0 and 1: a probability for a yes/no output



tanh

between -1 and 1: common inside older networks, such as LSTMs

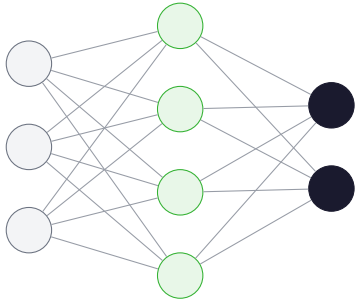


ReLU

$\max(0, z)$: the default for hidden layers; simple and trains well

Why not skip the activation? Without a non-linear f, stacking layers is pointless: a linear function of a linear function is still linear. Ten layers would be no more expressive than one.

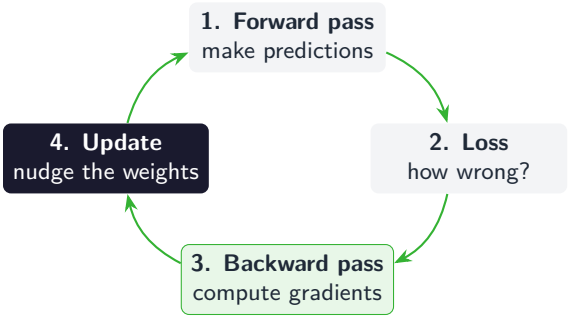
From Neurons to Networks



- Neurons are organized in **layers**; every neuron in one layer feeds every neuron in the next
- **Hidden layers** learn new **representations** of the input, in which the task becomes easier
- A word embedding (Day 2) is exactly such a learned representation

The XOR test: output 1 when exactly one of two inputs is 1. No single neuron can learn it; one hidden layer of a few neurons can.

How a Network Learns



epoch	one pass over all training data
batch	examples per weight update
loss	a number for how wrong the predictions are
gradient	for every weight: which way makes the loss bigger
learning rate	how big each update step is
optimizer	the update rule (SGD, Adam)

Frameworks compute the gradients for you (**autograd**): in PyTorch, step 3 is one line, `loss.backward()`.

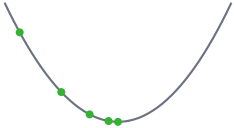
Gradient Descent and the Learning Rate

Training is like walking downhill in fog: you only feel the slope under your feet, so you take a step in the steepest downward direction, and repeat.



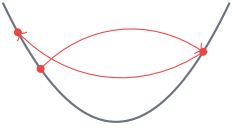
too small

tiny steps: training takes forever



about right

steady progress to the bottom

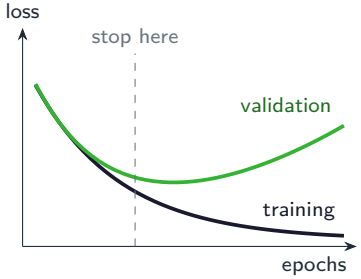


too large

overshoots, bounces around, or blows up

This is why fine-tuning BERT on Day 3 used a tiny learning rate (2e-5): the pre-trained weights are already good, and big steps would wreck what they learned.

Overfitting, and How to Fight It



Networks have many parameters, so they can **memorize** their training data, noise included. Training loss keeps falling; validation loss turns back up.

Dropout: randomly silence neurons during training

Weight decay: penalize large weights

Early stopping: stop when validation loss stops improving, and keep the best weights

More data: always the best remedy, rarely available

Three Stacks, One Model

The same network (one hidden layer of 64 neurons) in each stack:

scikit-learn

```
mlp = MLPClassifier((64,))
mlp.fit(X, y)
```

- ✔ just `fit` and `predict`
- ✔ fastest way to try one
- ✘ simple networks, no GPU

Keras

```
Sequential([
  Dense(64, "relu"),
  Dropout(0.5),
  Dense(4, "softmax")])
```

- ✔ stack layers, compile, fit
- ✔ concise and readable
- ✔ TensorFlow, JAX or PyTorch inside

PyTorch

```
for x, y in loader:
  loss = loss_fn(model(x), y)
  opt.zero_grad()
  loss.backward()
  opt.step()
```

- ✔ you write the training loop
- ✔ full control
- ✔ Hugging Face runs on it

When Are Neural Networks Worth It for Text?

A sobering result: on bag-of-words features, a neural network rarely does much better than logistic regression. The input already threw away word order and context, so the hidden layers have little extra structure to find.

Neural networks pay off when the **architecture** can use something a bag of words can't:

Local word order
convolutional networks
Bonus, Part 7

Sequences, memory
recurrent networks, LSTMs
Bonus, Part 8

Context + pre-training
transformers, BERT
Day 3

In the Notebook

- One neuron, and the activation functions, in a few lines of NumPy
- A network that learns XOR, **written from scratch**, backward pass included
- The same network with PyTorch's **autograd**
- One bill-title classifier, built in **scikit-learn, Keras and PyTorch**, side by side
- Provoking overfitting on purpose, and fixing it with dropout and early stopping
- Comparing everything with Day 3's baseline, on the same test set

`bonus/notebooks/6_neural_network_basics_exercises.ipynb` · 6 exercises · no GPU needed